

1-4 · Dockerfile 작성 및 이미지 최적화 · 이론 1H

Dockerfile 문법과 레이어

지시어 여섯 개와 레이어 캐시 — 순서 하나로 빌드가 몇 분에서 몇 초로

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

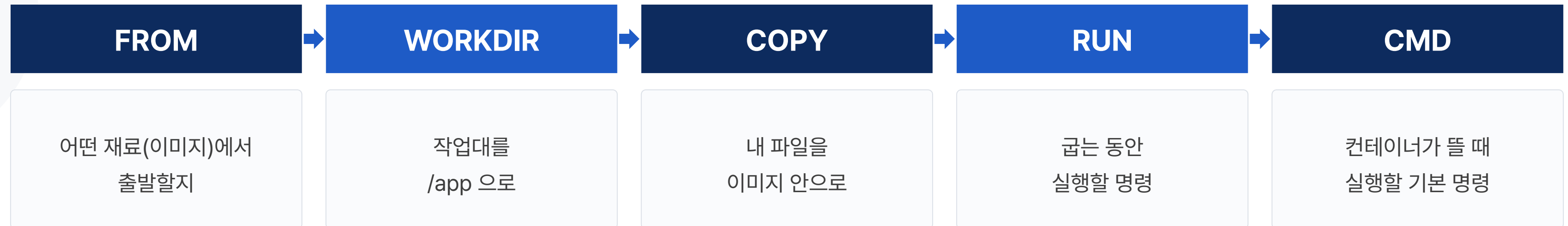
1시간 뒤에는

- ① Dockerfile 지시어 FROM·WORKDIR·COPY·RUN·ENV·CMD 의 역할을 말할 수 있다
- ② RUN(빌드 시점)과 CMD(실행 시점)를 구별한다
- ③ 레이어 캐시 원리로 「의존성 먼저, 소스 나중」 순서를 설명한다
- ④ slim·alpine 중 데이터 작업의 기본 베이스를 고른다

다음 1.5H 실습에서 이 순서를 일부러 틀려 보며 캐시가 깨지는 것을 확인합니다

1. 레시피 카드

Dockerfile 은 위에서부터 한 줄씩 실행되는 레시피



ENV(환경변수) · EXPOSE(포트 문서화) · ENTRYPOINT(고정 명령)는 필요할 때 더한다

docker build 는 이 카드를 위에서부터 한 줄씩 실행해 이미지를 굽는다

1. 레시피 카드

지시어 여덟 개 — 하는 일과 예

지시어	하는 일	예
FROM	어떤 이미지에서 출발할지	FROM python:3.12-slim
WORKDIR	이후 명령의 기준 디렉터리	WORKDIR /app
COPY	내 컴퓨터 파일을 이미지 안으로	COPY requirements.txt .
RUN	이미지를 만드는 중에 실행	RUN pip install -r requirements.txt
ENV	환경변수 설정	ENV PYTHONUNBUFFERED=1
EXPOSE	쓰는 포트를 문서화(열지는 않음)	EXPOSE 8000
CMD	컨테이너 시작 때 실행할 기본 명령	CMD ["python", "app.py"]
ENTRYPOINT	항상 실행되는 고정 명령	ENTRYPOINT ["python"]

EXPOSE 는 포트를 「문서화」할 뿐 열지 않는다 — 실제로 여는 것은 `docker run -p`

RUN 과 CMD — 가장 많이 헷갈리는 둘

RUN — 빌드 시점

이미지를 만드는 동안 실행되고 결과가 이미지에 굳는다

- docker build 중에 실행
- pip install · apt-get 같은 「준비」
- 실행할 때마다 층이 하나 늘어난다

VS

CMD — 실행 시점

이미지를 만들 때는 실행되지 않고 컨테이너를 띄울 때 실행된다

- docker run 할 때 실행
- python app.py 같은 「시작」
- Dockerfile 에 하나만 유효(마지막 것)

2. 레이어

한 줄이 한 층 — 바뀐 층부터 아래가 전부 다시 만들어진다

1 이미지는 층으로 쌓인 샌드위치

FROM 이 맨 아래 빵, 그 위에 COPY·RUN 이 한 층씩 — 각 층은 「이전 층에서 달라진 것」만 담는다

2 어떤 층이 바뀌면

그 위의 층은 전부 다시 만든다 — 아래층이 달라졌으니 그 위에 쌓은 것도 유효하지 않다

3 그래서 순서가 곧 속도

잘 안 바뀌는 것을 먼저, 자주 바뀌는 것을 나중에 — 의존성 설치 → 소스 복사

빌드 로그에서 찾을 것

CACHED

= 캐시 재사용

캐시 전략 한 문장 — 의존성 먼저, 소스 나중

잘못된 순서

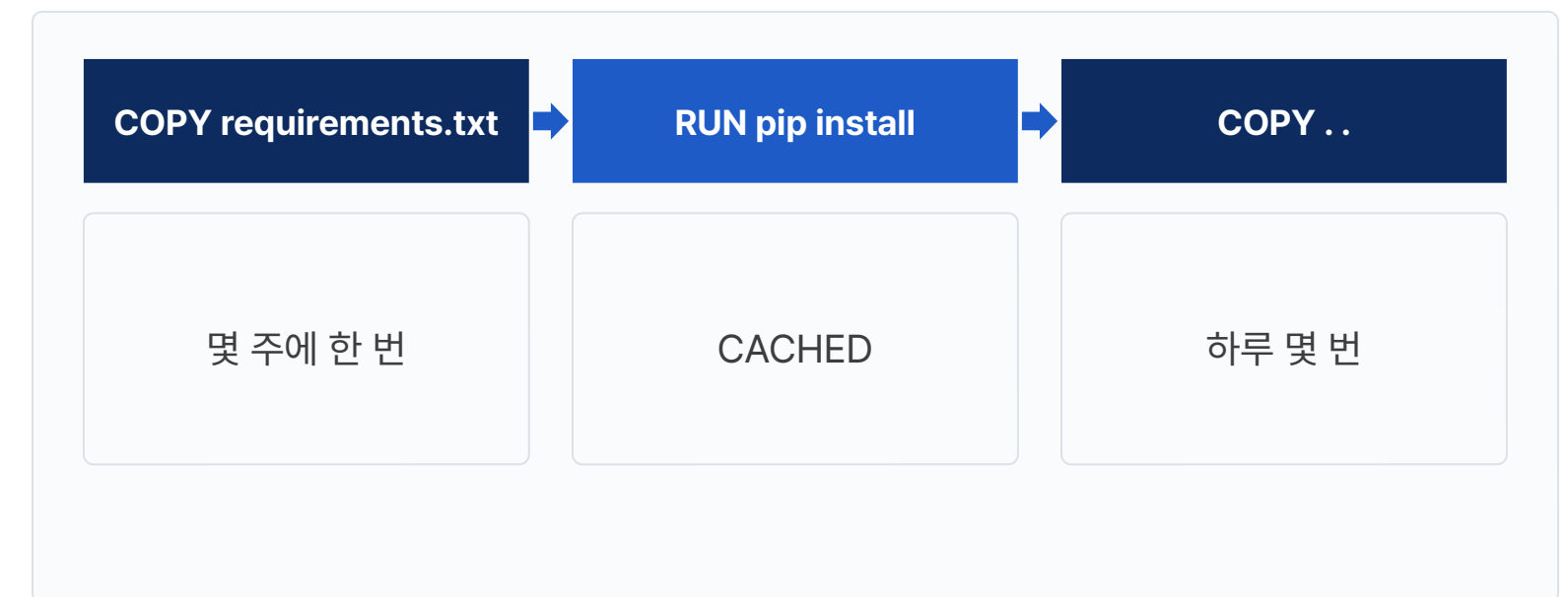
코드 한 글자만 고쳐도 pip install 이 처음부터 다시 돈다



VS

정답 순서

requirements.txt 가 그대로면 pip install 층은 CACHED



python:3.12 · slim · alpine — 무엇을 고를까

python:3.12 · 약 1GB

01

컴파일러·개발 도구가 다 들어 있다
빌드는 편하지만 무겁다

python:3.12-slim · 약 130MB

02

실행에 필요한 최소한
이번 과정의 기본값

python:3.12-alpine · 약 50MB

03

가장 작지만 C 라이브러리가 다르다
pandas·numpy 설치가 느리거나 실패

작다고 무조건 좋은 것이 아니다 — 데이터 작업은 slim 을 기본으로

4. 빌드 컨텍스트

`docker build -t data-app:0.1 .` — 마지막 점(.)의 정체

- 1 빌드 컨텍스트**
도커가 이미지를 만들 때 참고할 수 있는 폴더 — COPY 는 이 안의 파일만 가져온다
- 2 통째로 전송된다**
컨텍스트 폴더 내용이 도커 데몬으로 보내지므로 큰 파일이 섞이면 빌드가 느려진다
- 3 .dockerignore**
.git · __pycache__ · 데이터 파일을 적어 두면 전송에서 빠진다
- 4 컨텍스트 밖은 불가**
../다른폴더/파일 같은 경로를 COPY 할 수 없는 이유

이 1시간에서 꼭 가져갈 것

- 1 Dockerfile 은 위에서부터 한 줄씩 실행되고, 한 줄이 한 층이 된다
- 2 바뀐 층부터 아래(뒤)가 전부 다시 만들어진다
- 3 그래서 의존성 설치를 소스 복사보다 먼저 쓴다
- 4 데이터 작업의 기본 베이스는 slim 이다
- 5 COPY 는 빌드 컨텍스트(마지막 점) 안의 파일만 가져올 수 있다

1-4 · 이론 1H 마무리

다음은 이미지 빌드 실습 (1.5H)

파이썬 앱을 이미지로 굽고, 일부러 순서를 잘못 써 보며 캐시가 깨지는 것을 눈으로 확인합니다

✓ Dockerfile 을 직접 쓰고 docker build

✓ 빌드 로그에서 CACHED 확인

✓ 순서를 바꿔 캐시를 일부러 깨 보기

✓ .dockerignore 로 컨텍스트 줄이기

준비할 것

1-3 에서 만든 실습 폴더와 도커 데몬 실행 상태